

Basic Programs

1. Plot a Line Chart

Plot a simple line chart showing values [1, 2, 3, 4, 5] on the x-axis and [10, 20, 25, 30, 40] on the y-axis. Add labels and a title.

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y = [10, 20, 25, 30, 40]

plt.plot(x, y)
plt.xlabel("X-Axis")
plt.ylabel("Y-Axis")
plt.title("Simple Line Chart")
plt.show()
```

2. Plot a Bar Chart

Plot a bar chart for the following data:

Labels: ['Apples', 'Bananas', 'Cherries']

Values: [10, 15, 7]

```
import matplotlib.pyplot as plt

fruits = ['Apples', 'Bananas', 'Cherries']
quantity = [10, 15, 7]

plt.bar(fruits, quantity, color='skyblue')
plt.title("Fruit quantities")
plt.xlabel("Fruits")
plt.ylabel("quantity")
plt.show()
```

3. Plot a Pie Chart

Create a pie chart for the following data:

Labels = ['Math', 'Science', 'English', 'History']

Sizes = [30, 25, 25, 20]

```
Import matplotlib.pyplot as plt

subjects = ['Math', 'Science', 'English', 'History']
sizes = [30, 25, 25, 20]

plt.pie(sizes, labels=subjects, autopct='%1.1f%%', startangle=140)
plt.title("Subject Preference")
plt.show()
```

4. Add a Legend to a Plot

Plot two lines on the same graph:

- Line 1: $x = [1, 2, 3]$, $y = [2, 4, 6]$
 - Line 2: $x = [1, 2, 3]$, $y = [1, 2, 3]$
- Add a legend to distinguish the two lines.

```
import matplotlib.pyplot as plt

x = [1, 2, 3]
y1 = [2, 4, 6]
y2 = [1, 2, 3]

plt.plot(x, y1, label='Line 1')
plt.plot(x, y2, label='Line 2')
plt.legend()
plt.title("Two Lines with Legend")
plt.xlabel("X")
plt.ylabel("Y")
plt.show()
```

5. Save a Plot to File

Plot a bar chart and save it as an image file named "chart.png".

```
import matplotlib.pyplot as plt

x = ['A', 'B', 'C']
y = [5, 7, 3]

plt.bar(x, y, color='green')
plt.title("Sample Bar Chart")
plt.savefig("chart.png") # Saves the chart to current directory
plt.show()
```

6. Plot with Custom Line Style and Color

Plot a line with red dashed style ('r--') for $x = [1, 2, 3, 4]$ and $y = [2, 3, 5, 7]$.

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4]
y = [2, 3, 5, 7]

plt.plot(x, y, 'r--') # red dashed line
plt.title("Red Dashed Line")
plt.show()
```

7. Scatter Plot Example

Plot a scatter plot with:

```
x = [1, 2, 3, 4, 5]
```

```
y = [5, 4, 3, 2, 1]
```

```
Import matplotlib.pyplot as plt
```

```
x = [1, 2, 3, 4, 5]
```

```
y = [5, 4, 3, 2, 1]
```

```
plt.scatter(x, y, color='purple')
```

```
plt.title("Scatter Plot")
```

```
plt.xlabel("X")
```

```
plt.ylabel("Y")
```

```
plt.show()
```

8. Subplots Example

Create two plots (a line chart and a bar chart) side by side using `subplot`.

:

```
import matplotlib.pyplot as plt
```

```
x = [1, 2, 3, 4]
```

```
y1 = [1, 4, 9, 16]
```

```
y2 = [2, 3, 4, 5]
```

```
plt.subplot(1, 2, 1) # 1 row, 2 columns, 1st plot
```

```
plt.plot(x, y1)
```

```
plt.title("Line Chart")
```

```
plt.subplot(1, 2, 2) # 1 row, 2 columns, 2nd plot
```

```
plt.bar(x, y2)
```

```
plt.title("Bar Chart")
```

```
plt.tight_layout()
```

```
plt.show()
```

9. Plot a Histogram

Create a histogram from the data: [1, 2, 2, 3, 3, 3, 4, 4, 5]

```
import matplotlib.pyplot as plt
```

```
data = [1, 2, 2, 3, 3, 3, 4, 4, 5]
```

```
plt.hist(data, bins=5, color='orange', edgecolor='black')
plt.title("Histogram")
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.show()
```

10. Annotate a Point on a Plot

Plot a line chart and add a text label on the point (3, 9).

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y = [1, 4, 9, 16, 25]

plt.plot(x, y)
plt.annotate("This is 3,9", xy=(3, 9), xytext=(3, 15),
            arrowprops=dict(facecolor='black', shrink=0.05))
plt.title("Annotated Plot")
plt.show()
```

Pie Chart

1. Basic Pie Chart

Create a pie chart showing the following data:

- Labels: ['Apple', 'Banana', 'Cherry', 'Mango']
- Sizes: [30, 25, 20, 25]

```
import matplotlib.pyplot as plt

labels = ['Apple', 'Banana', 'Cherry', 'Mango']
sizes = [30, 25, 20, 25]

plt.pie(sizes, labels=labels)
plt.title("Fruit Distribution")
plt.show()
```

2. Add Percentage Display

Create a pie chart with labels and percentages shown on the slices using the same data as 1.

```
import matplotlib.pyplot as plt
```

```
labels = ['Apple', 'Banana', 'Cherry', 'Mango']
sizes = [30, 25, 20, 25]
```

```
plt.pie(sizes, labels=labels, autopct='%1.1f%%')
plt.title("Fruit Distribution with Percentages")
plt.show()
```

3. Explode a Slice

Highlight the 'Banana' slice by "exploding" it slightly out from the center.

```
import matplotlib.pyplot as plt

labels = ['Apple', 'Banana', 'Cherry', 'Mango']
sizes = [30, 25, 20, 25]
explode = [0, 0.1, 0, 0] # Only "Banana" is exploded

plt.pie(sizes, labels=labels, autopct='%1.1f%%', explode=explode)
plt.title("Exploded Pie Chart (Banana)")
plt.show()
```

4. Start from a Different Angle

Start the pie chart from 90 degrees instead of the default starting point.

```
import matplotlib.pyplot as plt

labels = ['Apple', 'Banana', 'Cherry', 'Mango']
sizes = [30, 25, 20, 25]

plt.pie(sizes, labels=labels, autopct='%1.1f%%', startangle=90)
plt.title("Pie Chart Starting at 90°")
plt.show()
```

5. Change Colors Manually

Set custom colors for each slice.

```
import matplotlib.pyplot as plt

labels = ['Apple', 'Banana', 'Cherry', 'Mango']
sizes = [30, 25, 20, 25]
colors = ['#ff9999', '#66b3ff', '#99ff99', '#ffcc99']

plt.pie(sizes, labels=labels, autopct='%1.1f%%', colors=colors,
        startangle=90)
plt.title("Pie Chart with Custom Colors")
plt.show()
```

Bar Chart

1. Basic Vertical Bar Chart

Plot a vertical bar chart for the following data:

- Categories: ['A', 'B', 'C', 'D']
- Values: [10, 15, 7, 12]

```
import matplotlib.pyplot as plt

categories = ['A', 'B', 'C', 'D']
values = [10, 15, 7, 12]

plt.bar(categories, values)
plt.title("Basic Vertical Bar Chart")
plt.xlabel("Categories")
plt.ylabel("Values")
plt.show()
```

2. Horizontal Bar Chart

Convert the chart in 1 into a **horizontal bar chart**.

```
import matplotlib.pyplot as plt

categories = ['A', 'B', 'C', 'D']
values = [10, 15, 7, 12]

plt.barh(categories, values)
plt.title("Horizontal Bar Chart")
plt.xlabel("Values")
plt.ylabel("Categories")
plt.show()
```

3. Change Bar Colors

Plot a bar chart with **custom colors** for each bar.

```
import matplotlib.pyplot as plt

categories = ['A', 'B', 'C', 'D']
values = [10, 15, 7, 12]
colors = ['red', 'green', 'blue', 'orange']
```

```
plt.bar(categories, values, color=colors)
plt.title("Bar Chart with Custom Colors")
plt.xlabel("Categories")
plt.ylabel("Values")
plt.show()
```

4. Add Labels Above Bars

Add **value labels** on top of each bar.

```
import matplotlib.pyplot as plt

categories = ['A', 'B', 'C', 'D']
values = [10, 15, 7, 12]

plt.bar(categories, values, color='skyblue')
plt.title("Bar Chart with Labels")
plt.xlabel("Categories")
plt.ylabel("Values")

# Add labels
for i in range(len(categories)):
    plt.text(i, values[i] + 0.5, str(values[i]), ha='center')

plt.show()
```

5. Compare Two Sets of Values (Grouped Bars)

Plot a bar chart comparing two sets of values for the same categories:

- Categories: ['1', '2', '3', '4']
- Sales 2024: [100, 150, 120, 180]
- Sales 2025: [130, 140, 160, 170]

```
import matplotlib.pyplot as plt
import numpy as np

categories = ['1', '2', '3', '4']
sales_2024 = [100, 150, 120, 180]
sales_2025 = [130, 140, 160, 170]

x = np.arange(len(categories))
width = 0.35

plt.bar(x - width/2, sales_2024, width, label='2024')
plt.bar(x + width/2, sales_2025, width, label='2025')

plt.xticks(x, categories)
plt.xlabel("uarter")
plt.ylabel("Sales")
plt.title("Sales Comparison 2024 vs 2025")
plt.legend()
plt.show()
```

Line Plot

1. Basic Line Plot

Plot a line chart for the following data:

- `x = [1, 2, 3, 4, 5]`
- `y = [10, 20, 15, 25, 30]`

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y = [10, 20, 15, 25, 30]

plt.plot(x, y)
plt.title("Basic Line Plot")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.show()
```

2. Line with Markers

Plot the same line as 1 but show **circle markers** on each data point.

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y = [10, 20, 15, 25, 30]

plt.plot(x, y, marker='o') # 'o' is the marker for circles
plt.title("Line Plot with Markers")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.show()
```

3. Multiple Lines in One Plot

Plot the following two lines on the same chart:

- `y1 = [10, 20, 15, 25, 30]`
- `y2 = [5, 15, 10, 20, 25]`

Use labels and a legend.

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
```

```

y1 = [10, 20, 15, 25, 30]
y2 = [5, 15, 10, 20, 25]

plt.plot(x, y1, label="Line 1", color='blue')
plt.plot(x, y2, label="Line 2", color='green')
plt.title("Two Lines in One Plot")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.legend()
plt.show()

```

4. Line with Custom Style and Color

Plot a line using a **red dashed line ('r--')** for this data:

- `x = [0, 1, 2, 3, 4]`
- `y = [0, 1, 4, 9, 16]`

```

import matplotlib.pyplot as plt

x = [0, 1, 2, 3, 4]
y = [0, 1, 4, 9, 16]

plt.plot(x, y, 'r--') # red dashed line
plt.title("Line with Red Dashed Style")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.show()

```

5. Line Plot with Grid and Axis Limits

Use the same data as 1, but:

- Add a **grid**
- Set x-axis limit from 0 to 6
- Set y-axis limit from 0 to 35

```

import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y = [10, 20, 15, 25, 30]

plt.plot(x, y, marker='o')
plt.title("Line Plot with Grid and Axis Limits")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.grid(True) # Add grid
plt.xlim(0, 6)
plt.ylim(0, 35)
plt.show()

```

Histogram

1. Basic Histogram

Plot a histogram of the following data:

```
data = [1, 2, 2, 3, 3, 3, 4, 4, 5]
```

```
import matplotlib.pyplot as plt
```

```
data = [1, 2, 2, 3, 3, 3, 4, 4, 5]
```

```
plt.hist(data)
plt.title("Basic Histogram")
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.show()
```

2. Histogram with Custom Number of Bins

Plot the same histogram with exactly **5 bins**.

```
import matplotlib.pyplot as plt
```

```
data = [1, 2, 2, 3, 3, 3, 4, 4, 5]
```

```
plt.hist(data, bins=5)
plt.title("Histogram with 5 Bins")
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.show()
```

3. Change Color and Add Edge Lines

Customize the histogram to use:

- Bar color: skyblue
- Black border for bars

```
import matplotlib.pyplot as plt
```

```
data = [1, 2, 2, 3, 3, 3, 4, 4, 5]
```

```
plt.hist(data, bins=5, color='skyblue', edgecolor='black')
plt.title("Styled Histogram")
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.show()
```

4. Histogram with Random Data

Generate 1000 random values from a normal distribution and plot a histogram with **20 bins**.

:

```
import matplotlib.pyplot as plt
import numpy as np

data = np.random.randn(1000) # 1000 normally distributed values

plt.hist(data, bins=20, color='purple', edgecolor='black')
plt.title("Histogram of Normal Distribution")
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.show()
```

5. Horizontal Histogram

Use the same basic data from 1 but display the histogram **horizontally**.

```
import matplotlib.pyplot as plt

data = [1, 2, 2, 3, 3, 3, 4, 4, 5]

plt.hist(data, bins=5, orientation='horizontal', color='lightgreen',
edgecolor='black')
plt.title("Horizontal Histogram")
plt.xlabel("Frequency")
plt.ylabel("Value")
plt.show()
```

Scatter Plot

1. Basic Scatter Plot

Plot a scatter plot with the following data:

- `x = [1, 2, 3, 4, 5]`
- `y = [5, 4, 3, 2, 1]`

```
import matplotlib.pyplot as plt
```

```
x = [1, 2, 3, 4, 5]  
y = [5, 4, 3, 2, 1]
```

```
plt.scatter(x, y)  
plt.title("Basic Scatter Plot")  
plt.xlabel("X-axis")  
plt.ylabel("Y-axis")  
plt.show()
```

2. Change Marker Color and Shape

Plot the same data but use:

- Marker shape: `^` (triangle)
- Color: `green`

```
import matplotlib.pyplot as plt
```

```
x = [1, 2, 3, 4, 5]  
y = [5, 4, 3, 2, 1]
```

```
plt.scatter(x, y, marker='^', color='green')  
plt.title("Scatter Plot with Triangle Markers")  
plt.xlabel("X-axis")  
plt.ylabel("Y-axis")  
plt.show()
```

3. Scatter Plot with Variable Point Sizes

Make point sizes vary using this list:

```
sizes = [20, 40, 60, 80, 100]
```

```
import matplotlib.pyplot as plt
```

```
x = [1, 2, 3, 4, 5]  
y = [5, 4, 3, 2, 1]  
sizes = [20, 40, 60, 80, 100]
```

```
plt.scatter(x, y, s=sizes, color='blue')
plt.title("Scatter Plot with Varying Sizes")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.show()
```

4. Scatter Plot with Two Groups of Points

Plot two different groups on the same scatter plot:

- Group 1: $x_1 = [1, 2, 3]$, $y_1 = [2, 4, 6]$
- Group 2: $x_2 = [1, 2, 3]$, $y_2 = [1, 2, 3]$

```
import matplotlib.pyplot as plt

x1 = [1, 2, 3]
y1 = [2, 4, 6]

x2 = [1, 2, 3]
y2 = [1, 2, 3]

plt.scatter(x1, y1, color='blue', label='Group 1')
plt.scatter(x2, y2, color='red', label='Group 2')
plt.title("Scatter Plot with Two Groups")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.legend()
plt.show()
```

5. Add Grid and Customize Axis Limits

:

Plot a scatter plot with:

- Grid enabled
- x-axis limit from 0 to 6
- y-axis limit from 0 to 6

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y = [5, 3, 1, 4, 2]

plt.scatter(x, y, color='purple')
plt.title("Scatter Plot with Grid and Axis Limits")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.grid(True)
plt.xlim(0, 6)
plt.ylim(0, 6)
plt.show()
```

Box Plot

1. Basic Box Plot

Create a basic box plot for the following data:

```
data = [7, 15, 13, 18, 16, 11, 10, 9, 21, 14]
```

```
import matplotlib.pyplot as plt
```

```
data = [7, 15, 13, 18, 16, 11, 10, 9, 21, 14]
```

```
plt.boxplot(data)
plt.title("Basic Box Plot")
plt.ylabel("Values")
plt.show()
```

2. Horizontal Box Plot

Plot the same data as a **horizontal box plot**.

```
import matplotlib.pyplot as plt
```

```
data = [7, 15, 13, 18, 16, 11, 10, 9, 21, 14]
```

```
plt.boxplot(data, vert=False)
plt.title("Horizontal Box Plot")
plt.xlabel("Values")
plt.show()
```

3. Box Plot for Multiple Datasets

Create a box plot comparing two datasets:

- group1 = [12, 14, 16, 18, 20]
- group2 = [8, 10, 12, 14, 16]

```
import matplotlib.pyplot as plt
```

```
group1 = [12, 14, 16, 18, 20]
group2 = [8, 10, 12, 14, 16]
```

```
data = [group1, group2]
```

```
plt.boxplot(data, labels=["Group 1", "Group 2"])
plt.title("Box Plot for Two Groups")
plt.ylabel("Values")
plt.show()
```

4. Show Outliers in a Box Plot

Create a box plot for a dataset with an **outlier**:

```
data = [5, 7, 8, 9, 10, 10, 11, 12, 13, 100]

import matplotlib.pyplot as plt

data = [5, 7, 8, 9, 10, 10, 11, 12, 13, 100]

plt.boxplot(data)
plt.title("Box Plot Showing Outlier")
plt.ylabel("Values")
plt.show()
```

Note: The point 100 will appear as an outlier in the plot.

5. Change Box Colors Using Patch Artist

Create a colored box plot for three datasets.

```
import matplotlib.pyplot as plt

data1 = [10, 12, 14, 16, 18]
data2 = [8, 9, 10, 11, 12]
data3 = [5, 6, 7, 8, 9]

data = [data1, data2, data3]

box = plt.boxplot(data, patch_artist=True, labels=["A", "B", "C"])

colors = ['lightblue', 'lightgreen', 'lightpink']
for patch, color in zip(box['boxes'], colors):
    patch.set_facecolor(color)

plt.title("Colored Box Plots for 3 Groups")
plt.ylabel("Values")
plt.show()
```

Area Chart

1. Basic Area Chart

Create an area chart for the following data:

- `x = [1, 2, 3, 4, 5]`
- `y = [2, 3, 5, 1, 4]`

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y = [2, 3, 5, 1, 4]

plt.fill_between(x, y, color='skyblue', alpha=0.5)
plt.plot(x, y, color='blue') # Optional line
plt.title("Basic Area Chart")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.show()
```

2. Area Chart with Labels and Grid

Use the same data as 1, and:

- Add axis labels
- Add a title
- Show a grid

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y = [2, 3, 5, 1, 4]

plt.fill_between(x, y, color='lightgreen', alpha=0.6)
plt.plot(x, y, color='green')
plt.title("Area Chart with Labels and Grid")
plt.xlabel("Time")
plt.ylabel("Value")
plt.grid(True)
plt.show()
```

3. Area Chart with Two Lines (Stacked)

Plot an area chart comparing two datasets (stacked):

- `y1 = [2, 3, 4, 5, 6]`
- `y2 = [1, 2, 1, 2, 1]`

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
```

```

y1 = [2, 3, 4, 5, 6]
y2 = [1, 2, 1, 2, 1]

plt.stackplot(x, y1, y2, labels=['Dataset 1', 'Dataset 2'],
              colors=['skyblue', 'lightcoral'])
plt.title("Stacked Area Chart")
plt.xlabel("X-axis")
plt.ylabel("Values")
plt.legend(loc='upper left')
plt.show()

```

4. Cumulative Area Chart

Plot a cumulative area chart where each new layer builds on top of the previous:

- `y1 = [1, 2, 3, 4, 5]`
- `y2 = [2, 2, 2, 2, 2]`

```

import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y1 = [1, 2, 3, 4, 5]
y2 = [2, 2, 2, 2, 2]

plt.stackplot(x, y1, y2, labels=['Base', 'Cumulative Layer'],
              colors=['#a1c4fd', '#c2e9fb'])
plt.title("Cumulative Area Chart")
plt.xlabel("X-axis")
plt.ylabel("Values")
plt.legend(loc='upper left')
plt.show()

```

5. Area Chart with Transparency (Alpha)

Use transparency (alpha) to plot overlapping area charts:

- `y1 = [1, 3, 4, 2, 5]`
- `y2 = [2, 1, 3, 4, 2]`

```

import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y1 = [1, 3, 4, 2, 5]
y2 = [2, 1, 3, 4, 2]

plt.fill_between(x, y1, color='orange', alpha=0.5, label='Series 1')
plt.fill_between(x, y2, color='blue', alpha=0.5, label='Series 2')
plt.title("Overlapping Area Charts with Transparency")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.legend()
plt.show()

```

Bar chart with individual bar colors

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots()

fruits = ['apple', 'blueberry', 'cherry', 'orange']
counts = [40, 100, 30, 55]
bar_labels = ['red', 'blue', '_red', 'orange']
bar_colors = ['tab:red', 'tab:blue', 'tab:red', 'tab:orange']

ax.bar(fruits, counts, label=bar_labels, color=bar_colors)

ax.set_ylabel('fruit supply')
ax.set_title('Fruit supply by kind and color')
ax.legend(title='Fruit color')

plt.show()
```

